

Training

Before running: Runtime → Change runtime type → T4 GPU

Cell 1 — Setup

```
from google.colab import drive
drive.mount('/content/drive')

!git clone https://github.com/prajwl7676/mini-clip-DLAI.git

%cd /content/mini-clip-DLAI
!pip install -q -r requirements.txt

import sys, os
sys.path.insert(0, '/content/mini-clip-DLAI')
```

```
Mounted at /content/drive
Cloning into 'mini-clip-DLAI'...
remote: Enumerating objects: 41, done.
remote: Counting objects: 100% (41/41), done.
remote: Compressing objects: 100% (34/34), done.
remote: Total 41 (delta 11), reused 35 (delta 5), pack-reused 0 (from 0)
Receiving objects: 100% (41/41), 6.91 MiB | 16.18 MiB/s, done.
Resolving deltas: 100% (11/11), done.
/content/mini-clip-DLAI
_____ 44.4/44.4 kB 4.7 MB/s eta 0:00:00
_____ 61.0/61.0 kB 6.4 MB/s eta 0:00:00
_____ 57.6/57.6 kB 6.3 MB/s eta 0:00:00
_____ 906.4/906.4 MB 1.7 MB/s eta 0:00:00
_____ 7.2/7.2 MB 91.6 MB/s eta 0:00:00
_____ 9.7/9.7 MB 117.9 MB/s eta 0:00:00
_____ 480.6/480.6 kB 34.3 MB/s eta 0:00:00
_____ 4.5/4.5 MB 114.0 MB/s eta 0:00:00
_____ 18.0/18.0 MB 91.4 MB/s eta 0:00:00
_____ 8.3/8.3 MB 96.3 MB/s eta 0:00:00
_____ 78.3/78.3 kB 8.0 MB/s eta 0:00:00
_____ 12.9/12.9 MB 104.4 MB/s eta 0:00:00
_____ 363.4/363.4 MB 4.1 MB/s eta 0:00:00
_____ 13.8/13.8 MB 102.1 MB/s eta 0:00:00
_____ 24.6/24.6 MB 74.0 MB/s eta 0:00:00
_____ 883.7/883.7 kB 61.5 MB/s eta 0:00:00
_____ 664.8/664.8 MB 2.1 MB/s eta 0:00:00
_____ 211.5/211.5 MB 5.7 MB/s eta 0:00:00
_____ 56.3/56.3 MB 14.8 MB/s eta 0:00:00
_____ 127.9/127.9 MB 7.9 MB/s eta 0:00:00
_____ 207.5/207.5 MB 7.4 MB/s eta 0:00:00
_____ 188.7/188.7 MB 7.0 MB/s eta 0:00:00
_____ 21.1/21.1 MB 62.4 MB/s eta 0:00:00
_____ 99.1/99.1 kB 7.6 MB/s eta 0:00:00
_____ 6.2/6.2 MB 79.5 MB/s eta 0:00:00
_____ 209.6/209.6 MB 6.2 MB/s eta 0:00:00
_____ 179.3/179.3 kB 15.9 MB/s eta 0:00:00
_____ 566.4/566.4 kB 41.1 MB/s eta 0:00:00
_____ 3.1/3.1 MB 51.2 MB/s eta 0:00:00
```

```
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is t
opencv-contrib-python 4.13.0.92 requires numpy>=2; python_version >= "3.9", but you have numpy 1.26.4 which is incompatible.
tobler 0.14.0 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.
tobler 0.14.0 requires tqdm>=4.67, but you have tqdm 4.66.6 which is incompatible.
jaxlib 0.7.2 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.
gcsfs 2025.3.0 requires fsspec==2025.3.0, but you have fsspec 2024.9.0 which is incompatible.
dataproc-spark-connect 1.1.0 requires tqdm>=4.67, but you have tqdm 4.66.6 which is incompatible.
cupy-cuda12x 14.0.1 requires numpy<2.6,>=2.0, but you have numpy 1.26.4 which is incompatible.
opencv-python 4.13.0.92 requires numpy>=2; python_version >= "3.9", but you have numpy 1.26.4 which is incompatible.
umap-learn 0.5.12 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.
tiffle 2026.4.11 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.
xarray-einstats 0.10.0 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.
rasterio 1.5.0 requires numpy>=2, but you have numpy 1.26.4 which is incompatible.
shap 0.52.0 requires numpy>=2, but you have numpy 1.26.4 which is incompatible.
pytensor 2.38.3 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.
opencv-python-headless 4.13.0.92 requires numpy>=2; python_version >= "3.9", but you have numpy 1.26.4 which is incompatible.
hdbscan 0.8.44 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.
jax 0.7.2 requires numpy>=2.0, but you have numpy 1.26.4 which is incompatible.
```

```
%cd /content/mini-clip-DLAI
```

```
/content/mini-clip-DLAI
```

```
import torch
import matplotlib.pyplot as plt
import numpy as np
from datasets import load_dataset
from transformers import DistilBertTokenizer

from src.model_architecture import CLIPModel
from src.dataset import FlickrDataset, get_transform, build_loaders
from src.loss import clip_loss
from src.train import train_epoch, validate, save_checkpoint, load_checkpoint

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device : {device}')
if device.type == 'cuda':
    print(f'GPU : {torch.cuda.get_device_name(0)}')
    print(f'VRAM : {torch.cuda.get_device_properties(0).total_memory / 1e9:.1f} GB')

Device : cuda
GPU : Tesla T4
VRAM : 15.6 GB
```

Cell 2 — Configuration

```
import os
# Hyperparameters
CFG = {
    # Data
    'total_samples': 10_000,
    'n_train': 8_000,
    'n_val': 1_000,
    # n_test = total - n_train - n_val = 1000

    # Model
    'embedding_dim': 256,
    'max_length': 77,

    # Training
    'batch_size': 64,
    'num_epochs': 15,
    'lr': 1e-4, # learning rate for encoders
    'lr_head': 1e-3, # higher lr for projection heads
    'weight_decay': 1e-2,
    'max_grad_norm': 1.0,
    'num_workers': 2,

    # Paths
    'checkpoint_dir': '/content/drive/MyDrive/mini-clip-dlai/checkpoints',
    'bestckpt_path': '/content/drive/MyDrive/mini-clip-dlai/checkpoints/best.pt',
}

os.makedirs(CFG['checkpoint_dir'], exist_ok=True)
print('Config ready.')
for k, v in CFG.items():
    print(f' {k:<20} : {v}')
```

```
Config ready.
total_samples      : 10000
n_train            : 8000
n_val              : 1000
embedding_dim      : 256
max_length         : 77
batch_size         : 64
num_epochs         : 15
lr                 : 0.0001
lr_head            : 0.001
weight_decay       : 0.01
max_grad_norm      : 1.0
num_workers        : 2
```

```
checkpoint_dir      : /content/drive/MyDrive/mini-clip-dlai/checkpoints
best_ckpt_path     : /content/drive/MyDrive/mini-clip-dlai/checkpoints/best.pt
```

Cell 3 — Load data

```
print('Loading Flickr30k (cached after first run)...')
full_ds = load_dataset('nlphuji/flickr30k', split='test')

tokenizer = DistilBertTokenizer.from_pretrained('distilbert-base-uncased')

# Split
n_train = CFG['n_train']
n_val   = CFG['n_val']
n_total = CFG['total_samples']

train_hf = full_ds.select(range(0, n_train))
val_hf   = full_ds.select(range(n_train, n_train + n_val))
test_hf  = full_ds.select(range(n_train + n_val, n_total))

# Datasets
train_ds = FlickrDataset(train_hf, tokenizer, get_transform(train=True))
val_ds   = FlickrDataset(val_hf,   tokenizer, get_transform(train=False))
test_ds  = FlickrDataset(test_hf,  tokenizer, get_transform(train=False))

# DataLoaders
train_loader, val_loader, test_loader = build_loaders(
    train_ds, val_ds, test_ds,
    batch_size = CFG['batch_size'],
    num_workers = CFG['num_workers'],
)

print(f'Train : {len(train_ds):,} samples → {len(train_loader)} batches')
print(f'Val   : {len(val_ds):,} samples → {len(val_loader)} batches')
print(f'Test  : {len(test_ds):,} samples → {len(test_loader)} batches')
```

```
Loading Flickr30k (cached after first run)...
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as an environment variable 'HF_TOKEN' or add it to your Colab secrets.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
README.md: 100%                               641/641 [00:00<00:00, 59.2kB/s]

flickr30k.py:      2.51k/? [00:00<00:00, 219kB/s]

TEST/test/0000.parquet: 100%                   506M/506M [00:04<00:00, 136MB/s]
TEST/test/0001.parquet: 100%                   502M/502M [00:06<00:00, 89.0MB/s]
TEST/test/0002.parquet: 100%                   506M/506M [00:03<00:00, 168MB/s]
TEST/test/0003.parquet: 100%                   512M/512M [00:04<00:00, 121MB/s]
TEST/test/0004.parquet: 100%                   504M/504M [00:03<00:00, 188MB/s]
TEST/test/0005.parquet: 100%                   495M/495M [00:04<00:00, 102MB/s]
TEST/test/0006.parquet: 100%                   495M/495M [00:06<00:00, 84.7MB/s]
TEST/test/0007.parquet: 100%                   497M/497M [00:08<00:00, 64.4MB/s]
TEST/test/0008.parquet: 100%                   289M/289M [00:05<00:00, 50.9MB/s]
Generating test split: 100%                   31014/31014 [00:46<00:00, 1384.79 examples/s]
tokenizer_config.json: 100%                   48.0/48.0 [00:00<00:00, 3.81kB/s]
vocab.txt:      232k/? [00:00<00:00, 3.54MB/s]
tokenizer.json:  466k/? [00:00<00:00, 3.49MB/s]
config.json: 100%                             483/483 [00:00<00:00, 48.1kB/s]
Train : 8,000 samples → 125 batches
Val   : 1,000 samples → 16 batches
Test  : 1,000 samples → 16 batches
```

Cell 4 — Build model and optimizer

- Encoder weights (pretrained) → small lr `1e-4`
- Projection heads (randomly initialised) → larger lr `1e-3`
- Temperature τ (scalar) → same as projection heads

```
model = CLIPModel(embedding_dim=CFG['embedding_dim']).to(device)

# Separate parameter groups with different learning rates
projection_params = (
    list(model.image_encoder.projection.parameters()) +
    list(model.text_encoder.projection.parameters()) +
    [model.log_temperature]
)
encoder_params = [
    p for p in model.parameters()
    if p.requires_grad and not any(p is pp for pp in projection_params)
]

optimizer = torch.optim.AdamW([
    {'params': encoder_params, 'lr': CFG['lr'], 'weight_decay': CFG['weight_decay']},
    {'params': projection_params, 'lr': CFG['lr_head'], 'weight_decay': 0.0},
])

# Cosine annealing: smoothly reduces lr from initial value → 0 over training
# This helps the model settle into a good minimum in the final epochs
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer,
    T_max=CFG['num_epochs'],
    eta_min=1e-6,
)

# Count parameters
total = sum(p.numel() for p in model.parameters())
trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f'Total params : {total:,}')
print(f'Trainable params : {trainable:,} ({100*trainable/total:.1f}%)')
print(f'Optimizer ready with {len(optimizer.param_groups)} param groups.')
```

Downloading: "<https://download.pytorch.org/models/resnet50-0676ba61.pth>" to /root/.cache/torch/hub/checkpoints/resnet50-0676ba61.pth [97.8M/97.8M [00:00<00:00, 118MB/s]]

model.safetensors: 100%

268M/268M [00:01<00:00, 198MB/s]

Total params : 90,592,833

Trainable params : 44,038,145 (48.6%)

Optimizer ready with 2 param groups.

Cell 5 — Resume from checkpoint (if exists)

```
loss_history = {'train': [], 'val': []}
start_epoch = 0
best_val_loss = float('inf')

# Look for the most recent epoch checkpoint
resume_path = None
for ep in range(CFG['num_epochs'] - 1, -1, -1):
    candidate = os.path.join(CFG['checkpoint_dir'], f'epoch_{ep:02d}.pt')
    if os.path.exists(candidate):
        resume_path = candidate
        break

if resume_path:
    print(f'Resuming from: {resume_path}')
    ckpt = load_checkpoint(model, optimizer, resume_path, device)
    start_epoch = ckpt['epoch'] + 1
    loss_history = ckpt['loss_history']
    best_val_loss = min(loss_history['val']) if loss_history['val'] else float('inf')
    print(f'Resumed from epoch {ckpt["epoch"]}. Continuing from epoch {start_epoch}.')
    print(f'Best val loss so far: {best_val_loss:.4f}')
```

```
else:
    print('No checkpoint found - starting training from scratch.')
```

No checkpoint found - starting training from scratch.

✓ Cell 6 — Training loop

```
import math

print(f'Starting training from epoch {start_epoch} / {CFG["num_epochs"]-1}')
print(f'Random loss baseline : log({CFG["batch_size"]}) = {math.log(CFG["batch_size"]):.3f}')
print('=' * 65)

for epoch in range(start_epoch, CFG['num_epochs']):

    # Train
    train_metrics = train_epoch(
        model, train_loader, optimizer, device,
        max_grad_norm=CFG['max_grad_norm'],
    )

    # Validate
    val_metrics = validate(model, val_loader, device)

    # Step scheduler
    scheduler.step()
    current_lr = scheduler.get_last_lr()[0]

    # Record loss history -
    train_loss = train_metrics['loss']
    val_loss = val_metrics['loss']
    loss_history['train'].append(train_loss)
    loss_history['val'].append(val_loss)

    # Print epoch summary
    improved = ' ← best' if val_loss < best_val_loss else ''
    print(
        f'Epoch {epoch+1:02d}/{CFG["num_epochs"]:02d} | '
        f'train {train_loss:.4f} | '
        f'val {val_loss:.4f} | '
        f't {train_metrics["temperature"]:.4f} | '
        f'lr {current_lr:.2e}'
        f'{improved}'
    )

    # Save per-epoch checkpoint
    epoch_ckpt = os.path.join(CFG['checkpoint_dir'], f'epoch_{epoch:02d}.pt')
    save_checkpoint(
        model, optimizer, epoch,
        train_loss, val_loss, loss_history,
        save_path=epoch_ckpt,
    )

    # Save best checkpoint
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        save_checkpoint(
            model, optimizer, epoch,
            train_loss, val_loss, loss_history,
            save_path=CFG['best_ckpt_path'],
        )

print('=' * 65)
print(f'Training complete. Best val loss: {best_val_loss:.4f}')
print(f'Best checkpoint saved to: {CFG["best_ckpt_path"]}')

Starting training from epoch 0 / 14
Random loss baseline : log(64) = 4.159
=====
Epoch 01/15 | train 2.3245 | val 1.7621 | τ 0.0693 | lr 9.89e-05 ← best
Epoch 02/15 | train 1.3133 | val 1.6806 | τ 0.0653 | lr 9.57e-05 ← best
Epoch 03/15 | train 0.8863 | val 1.6155 | τ 0.0599 | lr 9.05e-05 ← best
Epoch 04/15 | train 0.6304 | val 1.7282 | τ 0.0550 | lr 8.36e-05
Epoch 05/15 | train 0.4468 | val 1.7181 | τ 0.0506 | lr 7.52e-05
```

Epoch 06/15	train 0.3297	val 1.7249	τ 0.0473	lr 6.58e-05
Epoch 07/15	train 0.2554	val 1.8324	τ 0.0447	lr 5.57e-05
Epoch 08/15	train 0.1937	val 1.7498	τ 0.0426	lr 4.53e-05
Epoch 09/15	train 0.1566	val 1.7827	τ 0.0410	lr 3.52e-05
Epoch 10/15	train 0.1168	val 1.8216	τ 0.0397	lr 2.58e-05
Epoch 11/15	train 0.0915	val 1.8089	τ 0.0387	lr 1.74e-05
Epoch 12/15	train 0.0765	val 1.8115	τ 0.0381	lr 1.05e-05
Epoch 13/15	train 0.0667	val 1.8111	τ 0.0377	lr 5.28e-06
Epoch 14/15	train 0.0609	val 1.7789	τ 0.0376	lr 2.08e-06
Epoch 15/15	train 0.0464	val 1.7702	τ 0.0375	lr 1.00e-06

=====

Training complete. Best val loss: 1.6155

Best checkpoint saved to: /content/drive/MyDrive/mini-clip-dlai/checkpoints/best.pt

✓ Cell 7 — Plot train vs val loss curves

```
epochs_ran = len(loss_history['train'])
x = range(1, epochs_ran + 1)

fig, ax = plt.subplots(figsize=(9, 5))

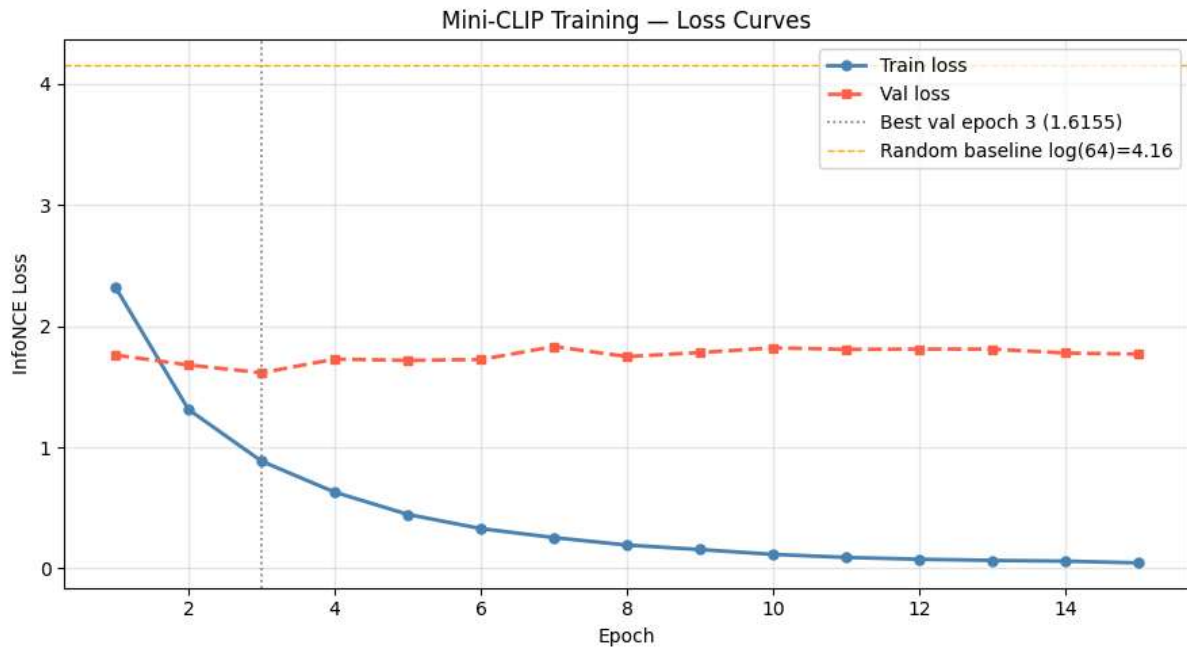
ax.plot(x, loss_history['train'], 'o-', color='steelblue',
        linewidth=2, markersize=5, label='Train loss')
ax.plot(x, loss_history['val'], 's--', color='tomato',
        linewidth=2, markersize=5, label='Val loss')

# Mark best epoch
best_epoch = int(np.argmin(loss_history['val'])) + 1
best_loss = min(loss_history['val'])
ax.axvline(best_epoch, color='grey', linestyle=':', linewidth=1.2,
          label=f'Best val epoch {best_epoch} ({best_loss:.4f})')

# Random baseline
baseline = math.log(CFG['batch_size'])
ax.axhline(baseline, color='orange', linestyle='--', linewidth=1,
          label=f'Random baseline log({CFG["batch_size"]})={baseline:.2f}')

ax.set_xlabel('Epoch')
ax.set_ylabel('InfoNCE Loss')
ax.set_title('Mini-CLIP Training - Loss Curves')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('assets/loss_curves.png', dpi=120, bbox_inches='tight')
plt.show()

print(f'Saved to assets/loss_curves.png')
print(f'Best epoch : {best_epoch}')
print(f'Best val : {best_loss:.4f}')
print(f'Train final : {loss_history["train"][-1]:.4f}')
```



Saved to assets/loss_curves.png

Best epoch : 3

Best val : 1.6155

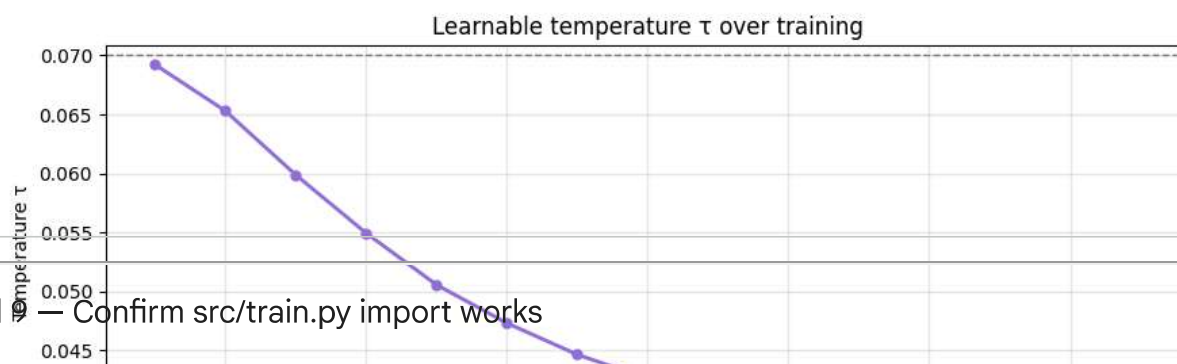
Train final : 0.0464

✓ Cell 8 — Temperature evolution over training

```
# Re-load all per-epoch checkpoints to extract temperature history
tau_history = []
for ep in range(epochs_ran):
    ckpt_path = os.path.join(CFG['checkpoint_dir'], f'epoch_{ep:02d}.pt')
    if os.path.exists(ckpt_path):
        ckpt = torch.load(ckpt_path, map_location='cpu')
        # Extract  $\tau$  from the saved model state
        log_tau = ckpt['model_state']['log_temperature']
        tau_history.append(log_tau.exp().item())

if tau_history:
    fig, ax = plt.subplots(figsize=(9, 4))
    ax.plot(range(1, len(tau_history)+1), tau_history,
            'o-', color='mediumpurple', linewidth=2, markersize=5)
    ax.axhline(0.07, color='grey', linestyle='--', linewidth=1, label='Init  $\tau=0.07$ ')
    ax.set_xlabel('Epoch')
    ax.set_ylabel('Temperature  $\tau$ ')
    ax.set_title('Learnable temperature  $\tau$  over training')
    ax.legend()
    ax.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig('assets/temperature_history.png', dpi=120, bbox_inches='tight')
    plt.show()
    print(f'Initial  $\tau$  : 0.0700')
    print(f'Final  $\tau$  : {tau_history[-1]:.4f}')
```

/tmp/ipykernel_7933/2354232651.py:6: FutureWarning: You are using `torch.load` with `weights\_only=False` (the current default ckpt = torch.load(ckpt_path, map_location='cpu'))



✓ Cell — Confirm src/train.py import works

```
from src.train import train_epoch, validate, save_checkpoint, load_checkpoint

# Load best checkpoint using the src helper
model_check = CLIPModel(embedding_dim=CFG['embedding_dim']).to(device)
projection_params = (
    list(model_check.image_encoder.projection.parameters()) +
    list(model_check.text_encoder.projection.parameters()) +
    [model_check.log_temperature]
)
encoder_params = [
    p for p in model_check.parameters()
    if p.requires_grad and not any(p is pp for pp in projection_params)
]

# 3. Initialize the optimizer with the exact same 2-group list structure
opt_check = torch.optim.AdamW([
    {'params': encoder_params, 'lr': CFG['lr']},
    {'params': projection_params, 'lr': CFG['lr_head']},
])

ckpt = load_checkpoint(model_check, opt_check, CFG['best_ckpt_path'], device)

print('src.train import works correctly.')
print(f'Best checkpoint epoch : {ckpt["epoch"] + 1}')
print(f'Best checkpoint val   : {ckpt["val_loss"]:.4f}')
print(f'Train history length   : {len(ckpt["loss_history"]["train"])} epochs')
```